

process what is process : A **process** is simply a **program that is currently running** on your computer. When a program runs, the operating system turns it into a process and gives it resources like:

- **CPU time** (to execute instructions)
- **Memory (RAM)** (to store data while running)
- **PID (Process ID)** (its unique number)
- **Open files / network connections**
- **Current state** (running, waiting, stopped, etc.)

thread what is thread : A **thread** is the **smallest unit of execution inside a process**.

All threads inside the same process share:

- Same memory (RAM)
- Same files
- Same resources

But each thread has its own:

- Instruction flow (what it is currently doing)
- Registers (small working data)
- Stack (function calls)

pcb A **PCB (Process Control Block)** is one of the most important data structures in an operating system. It is how the OS **keeps track of every process**.

✓ Track processes

The OS knows:

- which processes exist
- what each one is doing

✓ Manage CPU switching (very important)

When the CPU switches from one process to another (called **context switching**), the PCB stores everything needed to pause and resume a process.

posix thread A **POSIX thread** (usually called **Pthread**) is a **standard way to create and manage threads in Unix-like operating systems**.

pthread_create is a function from the POSIX Threads (pthreads) library that creates a new thread inside a process. Internally, it makes a system call (usually `clone()` in Linux) to the kernel, which creates the thread and makes it share the same address space (memory) with the parent process, while allocating a separate stack and thread-specific data for the new thread.

```
int pthread_create(  
    pthread_t *thread, => This is where the OS stores the thread ID  
    const pthread_attr_t *attr, => stack size, scheduling policy, detach state  
    void *(*start_routine)(void *), => This is the function the thread will run  
    void *arg => Data passed to the thread function  
);
```

pthread_join **waits for a thread to finish**, and optionally **collects its return value**.

What `pthread_mutex_init()` does

pthread_mutex_t It is a variable type used to create a lock that prevents multiple threads from accessing shared data at the same time. It is a complex structure that may contain:

- lock state (locked/unlocked)
- owner thread ID
- waiting threads queue
- kernel-related data

pthread_mutex_init It **initializes a mutex object** so it can be used for locking.

pthread_mutex_lock is a function used to **protect shared resources** in multithreading by making sure only **one thread at a time** can enter a critical section.

Mutex Waiting Queue When a thread tries to lock a mutex that is already locked, the kernel suspends that thread and places it into a waiting queue associated with the mutex (using the futex mechanism on Linux). The thread is marked as blocked, removed from CPU scheduling, and does not consume CPU time while waiting. When the mutex is eventually unlocked, the kernel selects one of the waiting threads, removes it from the queue, marks it as ready, and allows the scheduler to run it again so it can acquire the mutex and continue execution.

pthread_mutex_unlock is used to **release a mutex**, meaning it allows other threads to enter the critical section.

pthread_cond_t

atomic instruction An atomic instruction is a low-level CPU operation that executes entirely or not at all, appearing to other threads as an instantaneous, indivisible action. It cannot be interrupted or partially completed, making it crucial for synchronizing memory access in multithreaded systems without using slow lock-based primitives. Common examples include compare-and-swap (CAS), fetch-and-add, and test-and-set.

- **Compare-and-swap (CAS):** Updates a memory location only if it still contains a previously read value.
- **Fetch-and-add:** Adds a value to a memory location and returns the old value.
- **Atomic Swap:** Swaps a value in memory with a register value.